# Design OPTIONS to Inform Clients About Supported Operations

Explore how to implement the OPTIONS HTTP method in REST APIs to inform clients about supported operations. Understand its key features, request and response headers, and the role of CORS in enabling proper client-server communication, enhancing your API design and development skills.

## Inform clients about supported operations

A client should be able to query a service for the list of supported operations. In the world of REST APIs, that is achieved by issuing an OPTIONS request to the service. Conversely, a REST service uses the OPTIONS method to inform clients about the supported operations.

According to the specifications, “The OPTIONS method represents a request for information about the communication options available on the request/response chain identified by the Request-URI.”

According to the above specification, the OPTIONS method has the following features:

- It’s idempotent. That means multiple OPTIONS requests to the same URI with the same parameters will result in the same effect on the server as a single OPTIONS request.
- The response should provide all the operations supported by the servi
part of the response header.
- Body is optional for both the request and the response.

Ask

- A `200 OK` response will include any optional features implemented by the service.
- Responses are not cacheable.

Similar to GET or DELETE, OPTIONS request has an optional body, and similar to POST, it doesn't specify a specific resource ID.

## Request/response structure

Based on the above points, let's look at the request and response structure of an OPTIONS request.

### Request header

Request headers of OPTIONS are:

- **Method:** This is the HTTP method to be executed by the server. It'll be OPTIONS in this case.
- **URL/URI:** This is the identifying URI of the resource.
- **Version:** This is the version of the HTTP, i.e., 1.0, 1.1, and so on.
- **Accept:** This is the MIME type/format of the body of the response that the client will accept. This header is optional and rarely used. This is kept in case future requests need to specify the response body type.

Since the OPTIONS request doesn't have a body, let's look at the response headers.

### Response header

For an OPTIONS request, the response header status is mandatory. Any other server-specific operations supported by the service are optional headers. For example, Access-Control-Allow-Methods is one of the headers that are se specific.

Ask

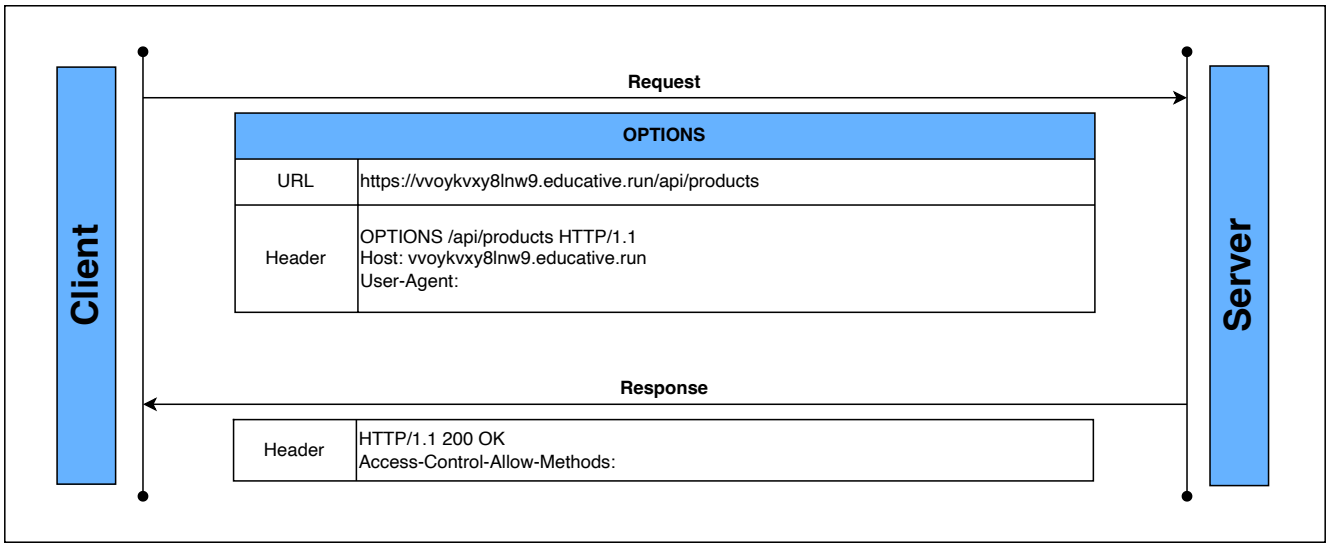
> **Note:** Browser preflight requests are HTTP OPTIONS requests based on CORS.

- **Status:** This is the status of the request processing. It should have a value of `200 OK` only if the headers inform the client of supported operations.
- **Content-Length (Optional):** This is the size of the content within the body. It's present only if the status is `200 OK`. In this case, it'll always be `0`.
- **Access-Control-Allow-Methods (Optional):** This is an optional header that is used in conjunction with CORS. This tells the client which methods are supported by the server and available for it to consume.

There can be other optional headers based on the service configuration.

> **Access-Control-Allow-Methods vs. Allow**
>
> There are two reasons why we use the `Access-Control-Allow-Methods` instead of the `Allow` header:
>
> 1. Allow is interpreted incorrectly by JavaScript and hence JavaScript REST API clients aren't able to understand what methods are supported by the API endpoints if only the `Allow` header is used. `Access-Control-Allow-Methods` is required for the correct functioning of JavaScript REST API clients that use preflight requests.
> 2. The default header provided by the frameworks is `Allow`. However, `Allow` only provides a way to tell what methods are allowed. `Access-Control-Allow-Methods`, coupled with other CORS headers, provides the client with a full overview of the capabilities of the API, including security features.

Ask




Request/response structure for OPTIONS API to inform clients about supported operations

## Products API

Based on the above points, the endpoint for informing clients about the operations supported by the products API will be as follows:

index.js Javascript (babel-node)

```
OPTIONS /api/products HTTP/1.1
Host: localhost:5000
User-Agent: curl/7.71.1
```

Ask

The `User-Agent` header will be optional.

The response header provided by the API will be:

index.js

Javascript (babel-node)

```
HTTP/1.1 200 OK
Access-Control-Allow-Methods: GET,POST,PUT,DELETE,OPTIONS
```

Ask